



Bilkent University

EEE 443 - Neural Networks

Third Assignment Report

Fall 2025–2026

Aleyna Yaren Öztürken - 22103420

November 21, 2025

Contents

1	Introduction	1
2	Methodology	1
2.1	Data Preprocessing	1
2.2	Network Architecture	1
2.3	Activation Functions	2
2.4	Backpropagation Algorithm	2
2.4.1	Online Learning	3
2.4.2	Batch Learning	4
3	Hyperparameters and Experimental Setup	5
4	Results and Discussion	6
4.1	Eight Hyperparameter Combinations with Online Learning	6
4.1.1	First Combination $n=400$, $\eta=0.01$, Case 1 Activation Function	6
4.1.2	Second Combination $n=400$, $\eta=0.03$, Case 1 Activation Function	8
4.1.3	Third Combination $n=400$, $\eta=0.01$, Case 2 Activation Function	10
4.1.4	Fourth Combination $n=400$, $\eta=0.03$, Case 2 Activation Function	12
4.1.5	Fifth Combination $n=800$, $\eta=0.01$, Case 1 Activation Function	14
4.1.6	Sixth Combination $n=800$, $\eta=0.03$, Case 1 Activation Function	16
4.1.7	Seventh Combination $n=800$, $\eta=0.03$, Case 1 Activation Function	18
4.1.8	Eighth Combination $n=800$, $\eta=0.03$, Case 2 Activation Function	20
4.2	Overall Analysis of Online Learning Experiments	22
4.3	Best Hyperparameter Combination Used in Batch Learning	23
4.3.1	Batch size $B=20$	23
4.3.2	Batch size $B=200$	25
5	Conclusion	26

1 Introduction

Handwritten digit recognition is a fundamental problem in the field of pattern recognition and machine learning. It serves as a basis for numerous real-world applications, such as automated mail sorting, check processing, and data entry. The goal is to train a model that can correctly identify a digit (0-9) from an input image. In this assignment, the well-known MNIST dataset is utilized, which is a standard benchmark for image classification algorithms. It consists of 60,000 training images and 10,000 testing images, each being a 28x28 pixel grayscale representation of a handwritten digit. This assignment focuses on understanding the core mechanics of the backpropagation algorithm and analyzing how different network configurations and hyperparameters affect learning and generalization performance.

2 Methodology

In this assignment the implementation of a two-layer feedforward neural network built from scratch to perform handwritten digit classification on the MNIST dataset. This network implementation had been done without using any special machine learning libraries as it is required. The network was trained using the online backpropagation algorithm. A series of experiments were conducted to analyze the impact of key hyperparameters, including the number of hidden neurons (400 and 800), the choice of activation functions (tanh and ReLU), and the learning rate (0.01 and 0.03). The performance of each configuration was evaluated based on training and test accuracy, error rates, and computational time. Further analysis includes a comparison with batch learning, discussion of convergence behavior, and evaluation of misclassification patterns through confusion matrices.

2.1 Data Preprocessing

Raw data is rarely in an optimal format for neural network training. Therefore, a preprocessing pipeline was established to transform the MNIST dataset into a structured and normalized format. This ensures that the network receives consistent input, which aids in stable and efficient learning. The specific preprocessing steps were as follows:

1. **Flattening (Vectorization):** The neural network's input layer is designed as a single vector. To accommodate this, each 2D image with dimensions 28x28 pixels was reshaped into a 1D vector of length 784 ($28 \times 28 = 784$).
2. **Normalization:** The pixel intensity values in the original dataset range from 0 (black) to 255 (white). To prevent issues with large input values and to ensure faster convergence of the learning algorithm, these values were scaled to a floating-point range of $[0, 1]$ by dividing each pixel's value by 255.
3. **Label Encoding:** The network's output layer consists of 10 neurons, requiring a target vector for the error calculation. The ground-truth labels (e.g., '5', '7') were converted into 10-dimensional target vectors. As specified in the assignment, for a given digit i , the neuron at the corresponding index has a target value of "+1", while all other nine neurons have a target value of "-1". The digit '0' was mapped to the 10th neuron (index 9).

2.2 Network Architecture

A standard multilayer perceptron with a single hidden layer was constructed. This architecture is capable of learning non-linear relationships between the input pixels and the output digit classes. The network consists of three distinct layers.

1. **Input Layer:** This layer acts as the entry point for the data. It contains 784 neurons, one for each pixel in the flattened input image. An additional bias node (threshold) with a constant value of 1 is appended to the input vector. This enables the network to learn a bias term, which provides a learnable offset for the activation function.
2. **Hidden Layer:** This intermediate layer is responsible for learning abstract features and complex patterns from the input data. The number of neurons, N , was a key hyperparameter, with values of 400 and 800 being tested. Like the input layer, a bias node is appended to the output of this layer before it is passed to the next stage.
3. **Output Layer:** This final layer produces the network's prediction. It contains 10 neurons, where each neuron's output activation corresponds to the network's confidence for one of the digits (1, 2, ..., 9, 0).
4. **Weight Initialization:** All weights connecting the layers were initialized using a random Gaussian distribution with a mean of 0 and a standard deviation of 0.1. This random initialization is critical to break symmetry and allow neurons to learn different features.

2.3 Activation Functions

Activation functions are applied to the weighted sum of inputs at each neuron, introducing the necessary non-linearity that allows the network to learn complex mappings. If activation functions were omitted, the entire network would collapse into a single linear model. Two experimental cases for activation functions were implemented:

1. **Case 1 (All Tanh):** The hyperbolic tangent function ($\tanh$) was used as the activation for both the hidden and output layers. This function squashes its input into the range $[-1, 1]$, which pairs well with the $[-1, +1]$ target encoding.

$$\tanh(v) = \frac{1 - e^{-2v}}{1 + e^{-2v}}$$

2. **Case 2 (ReLU/Tanh):** The Rectified Linear Unit (ReLU) was used for the hidden layer, while the $\tanh$ function was retained for the output layer. ReLU is a popular and computationally efficient function that helps mitigate the vanishing gradient problem. Hence it is often preferred in hidden layers.

$$\text{ReLU}(v) = \max(0, v)$$

2.4 Backpropagation Algorithm

Backpropagation is an efficient method for calculating the gradient of the loss function with respect to each weight and bias in the network. This gradient indicates the direction in which the weights should be adjusted to minimize the error. The algorithm works by applying the chain rule to iteratively compute these gradients, starting from the output layer and moving backward towards the input layer. The process for a single training pattern involves two main phases after an initial forward pass:

1. **Forward Pass:** The input pattern is fed through the network, and the output is calculated. All intermediate values (the weighted sums and activation outputs at each neuron) are stored.
2. **Backward Pass (Error Propagation):** This phase calculates an "error signal" (delta, δ) for each neuron, which quantifies how much that neuron contributed to the total error.

- The delta for an output neuron k is calculated as the difference between the desired output (d_k) and the actual output (o_k), multiplied by the derivative of the activation function (f'):

$$\delta_k = (d_k - o_k) \cdot f'(v_k)$$

where v_k is the weighted sum of inputs to neuron k .

- The delta for a hidden neuron j is calculated by taking the sum of the deltas from all neurons (k) in the next layer, weighted by the connections (w_{kj}), and then multiplying by the derivative of the hidden neuron's activation function:

$$\delta_j = \left(\sum_k \delta_k w_{kj} \right) \cdot f'(v_j)$$

3. **Weight Update:** Once the deltas are known, the gradient for each weight is calculated, and the weights are updated using gradient descent. The change in a weight w_{ij} (from neuron j to neuron i) is proportional to the delta of the destination neuron (δ_i) and the output of the source neuron (o_j), scaled by the learning rate η :

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \eta \cdot \delta_i \cdot o_j$$

2.4.1 Online Learning

Online learning is a procedure for training where it updates the neural network's parameters right after each training instance is fed into the system. A major benefit of this technique is the possibility of fast convergence, as the frequent modifications enable the model to assimilate information starting from the initial data point. This inherently random adjustment mechanism, which derives its gradient estimate from a single sample is beneficial for seeking global minima. A singular, noisy step can supply the necessary impulse to move out of a local minimum and towards a better solution, a maneuver stable methods might fail to execute. Conversely, this considerable variability in the gradient calculation is a significant drawback because it introduces instability into the learning progression. The calculated inaccuracy can oscillate with every adjustment, which may hinder the model from settling on the best solution. Instead, the parameters often fluctuate near the desired setting, making the precise calibration of the final result more difficult than with methods that employ a more consistent, averaged gradient.

The "online" training mode was used, for 8 hyperparameter combinations. After the getting the results of the online backpropagation algorithm, the hyperparameter combination which gave the best test accuracy was selected to be used with batch learning algorithm for comparison. The following gives the algorithmic steps of the online backpropagation method.

Algorithm 1: Online Learning Algorithm

Input: Training set $\mathcal{S} = \{(x^{(i)}, d^{(i)})\}_{i=1}^M$. Learning rate $\eta > 0$. Maximum epochs = 50.

Output: Final trained weight matrices $\mathbf{W}_{ih}$ (input-to-hidden) and $\mathbf{W}_{ho}$ (hidden-to-output).

- **Initialize:** Weight matrices $\mathbf{W}_{ih}$ and $\mathbf{W}_{ho}$ with small random values from a Gaussian distribution. These matrices include an extra column for bias/threshold weights.
- For each epoch from 1 to 50:
 - **Shuffle:** Randomize the order of the samples in the training set $\mathcal{S}$.
 - For each sample $(x^{(i)}, d^{(i)})$ in the shuffled training set:
 - * **1. Forward Propagation:**
 - Augment input vector with bias term: $x_{bias}^{(i)} \leftarrow [x^{(i)}; 1]$.
 - Calculate hidden layer net input: $\mathbf{v}_h \leftarrow \mathbf{W}_{ih} \cdot x_{bias}^{(i)}$.
 - Calculate hidden layer output: $\mathbf{o}_h \leftarrow f_h(\mathbf{v}_h)$.
 - Augment hidden output with bias term: $\mathbf{o}_{h,bias} \leftarrow [\mathbf{o}_h; 1]$.
 - Calculate output layer net input: $\mathbf{v}_o \leftarrow \mathbf{W}_{ho} \cdot \mathbf{o}_{h,bias}$.
 - Calculate final network output: $\mathbf{o}_o \leftarrow f_o(\mathbf{v}_o)$.
 - * **2. Backward Propagation:**
 - Calculate the error: $\mathbf{e} \leftarrow d^{(i)} - \mathbf{o}_o$.
 - Calculate output layer delta: $\delta_o \leftarrow \mathbf{e} \odot f'_o(\mathbf{v}_o)$.
 - Let $\mathbf{W}_{ho, no\ bias}$ be the weight matrix $\mathbf{W}_{ho}$ excluding the last (bias) column.
 - Propagate error to hidden layer: $\mathbf{e}_h \leftarrow (\mathbf{W}_{ho, no\ bias})^T \cdot \delta_o$.
 - Calculate hidden layer deltas: $\delta_h \leftarrow \mathbf{e}_h \odot f'_h(\mathbf{v}_h)$.
 - * **3. Update Weights (Online Update):**
 - Update hidden-to-output weights: $\mathbf{W}_{ho} \leftarrow \mathbf{W}_{ho} + \eta \cdot (\delta_o \cdot (\mathbf{o}_{h,bias})^T)$.
 - Update input-to-hidden weights: $\mathbf{W}_{ih} \leftarrow \mathbf{W}_{ih} + \eta \cdot (\delta_h \cdot (x_{bias}^{(i)})^T)$.
 - Output final weights $\mathbf{W}_{ih}$ and $\mathbf{W}_{ho}$.

2.4.2 Batch Learning

In the batch learning a network's weights are changed in a single, comprehensive update after processing the entire dataset. This adjustment is guided by the error gradient that is computed across all available training examples. This stands in sharp contrast to online learning, which performs numerous small updates based on one sample at a time. By averaging the gradient over every sample, the batch method provides a more stable approximation of the error surface's true gradient, promoting a smoother, more straightforward journey toward a local minimum. The main strength of this technique lies in its stability. Because the weight adjustments are not skewed by the peculiarities of individual data points, it avoids the pronounced oscillations that can characterize online learning. However, batch learning is not without its challenges. It is, first and foremost, computationally intensive and slow, as it must cycle through every piece of training data before making any correction to the weights, a process that is impractical for massive datasets. Furthermore, its very directness can be more susceptible to getting permanently lodged in a poor local minimum. The noisier update process of online learning can sometimes be an advantage.

Algorithm 2: Batch Learning Algorithm

Input: Training set $\mathcal{S} = \{(x^{(i)}, d^{(i)})\}_{i=1}^M$. Learning rate $\eta > 0$. Maximum epochs = 50.

Output: Final trained weight matrices $\mathbf{W}_{ih}$ (input-to-hidden) and $\mathbf{W}_{ho}$ (hidden-to-output).

- **Initialize:** Weight matrices $\mathbf{W}_{ih}$ and $\mathbf{W}_{ho}$ with small random values from a Gaussian distribution. These matrices include an extra column for bias/threshold weights.
- For each epoch from 1 to 50:
 - **1. Initialize Gradient Accumulators:** Set the total gradient matrices to zero at the start of the epoch.
 - * $\Delta \mathbf{W}_{ih} \leftarrow \mathbf{0}$.
 - * $\Delta \mathbf{W}_{ho} \leftarrow \mathbf{0}$.
 - **2. Accumulate Gradients over the Batch:**
 - For each sample $(x^{(i)}, d^{(i)})$ in the training set $\mathcal{S}$:
 - * **Forward Propagation:**
 - Augment input vector with bias term: $x_{bias}^{(i)} \leftarrow [x^{(i)}; 1]$.
 - Calculate hidden layer net input: $\mathbf{v}_h \leftarrow \mathbf{W}_{ih} \cdot x_{bias}^{(i)}$.
 - Calculate hidden layer output: $\mathbf{o}_h \leftarrow f_h(\mathbf{v}_h)$.
 - Augment hidden output with bias term: $\mathbf{o}_{h,bias} \leftarrow [\mathbf{o}_h; 1]$.
 - Calculate output layer net input: $\mathbf{v}_o \leftarrow \mathbf{W}_{ho} \cdot \mathbf{o}_{h,bias}$.
 - Calculate final network output: $\mathbf{o}_o \leftarrow f_o(\mathbf{v}_o)$.
 - * **Backward Propagation:**
 - Calculate the error: $\mathbf{e} \leftarrow d^{(i)} - \mathbf{o}_o$.
 - Calculate output layer delta: $\delta_o \leftarrow \mathbf{e} \odot f'_o(\mathbf{v}_o)$.
 - Let $\mathbf{W}_{ho, no\ bias}$ be the weight matrix $\mathbf{W}_{ho}$ excluding the last (bias) column.
 - Propagate error to hidden layer: $\mathbf{e}_h \leftarrow (\mathbf{W}_{ho, no\ bias})^T \cdot \delta_o$.
 - Calculate hidden layer deltas: $\delta_h \leftarrow \mathbf{e}_h \odot f'_h(\mathbf{v}_h)$.
 - * **Add Sample Gradients to Accumulators:**
 - $\Delta \mathbf{W}_{ho} \leftarrow \Delta \mathbf{W}_{ho} + (\delta_o \cdot (\mathbf{o}_{h,bias})^T)$.
 - $\Delta \mathbf{W}_{ih} \leftarrow \Delta \mathbf{W}_{ih} + (\delta_h \cdot (x_{bias}^{(i)})^T)$.
 - **3. Update Weights and Biases (Once per Epoch):** After processing all samples, update the network parameters using the average gradient.
 - * $\mathbf{W}_{ho} \leftarrow \mathbf{W}_{ho} + \eta \cdot \frac{1}{M} \Delta \mathbf{W}_{ho}$.
 - * $\mathbf{W}_{ih} \leftarrow \mathbf{W}_{ih} + \eta \cdot \frac{1}{M} \Delta \mathbf{W}_{ih}$.
 - Output final weights $\mathbf{W}_{ih}$ and $\mathbf{W}_{ho}$.

3 Hyperparameters and Experimental Setup

In this assignment, the experimental setup was set in Google Colab environment with using Jupyter Notebook. The all experiments done in that environment. Since the code format is required to be .py file and in order to sustain the consistence, the code is transformed to run in local environment. After the transformation the code is checked whether it is working. Online learning algorithm was

implemented for 8 different hyperparameter combinations. Afterwards the batch learning algorithm was implemented with the best hyperparameter combination which was founded as $n=800$, $\eta=0.01$ and Case 2 activation function.

- **Number of hidden neurons (n):** $\{400, 800\}$
- **Learning rate (η):** $\{0.01, 0.03\}$
- **Activation function cases:** $\{\text{Case 1: All Layers - tanh, Case 2: Hidden Layer - RELU, Output Layer - tanh}\}$

4 Results and Discussion

4.1 Eight Hyperparameter Combinations with Online Learning

4.1.1 First Combination $n=400$, $\eta=0.01$, Case 1 Activation Function

Metric	Value
<i>Configuration Parameters</i>	
Hidden Neurons	400
Activation Case	Case 1
Learning Rate (η)	0.01
Epochs Run	50
<i>Training Performance</i>	
Total Training Duration	3707.44 s
Average Time per Epoch	74.15 s
Final Training Error	0.0139
Final Training Accuracy	99.47%
<i>Test Performance</i>	
Final Test Error	0.0786
Final Test Accuracy	98.04%
Average Inference Time	0.1294 ms / pattern

Table 1: Performance Metrics for $n=400$, $\eta=0.01$, Case 1 Activation Function

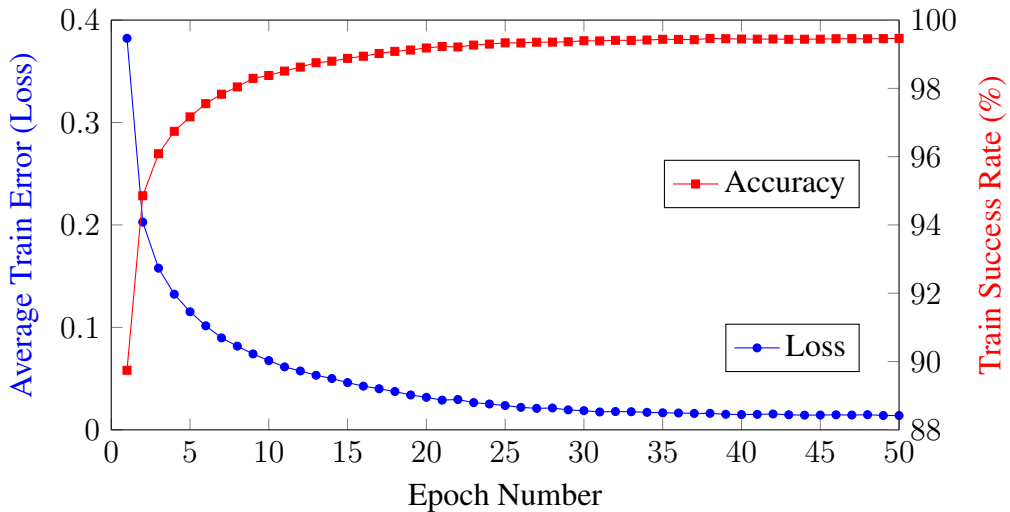


Figure 1: Online Training Performance: Loss and Accuracy vs. Epochs for $n=400$, $\eta=0.01$, Case 1

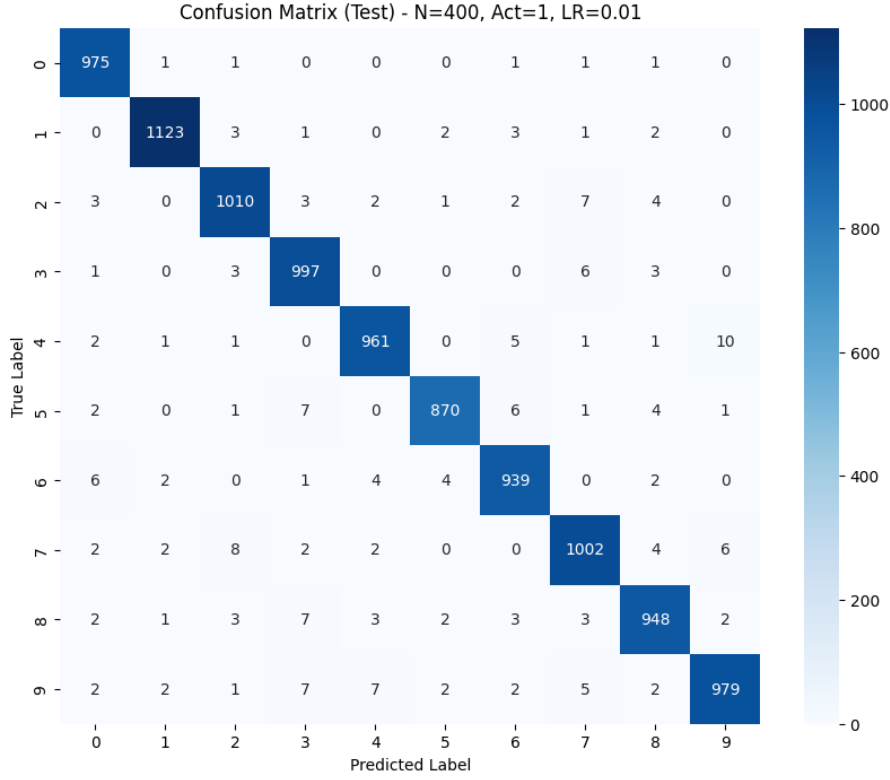


Figure 2: Confusion Matrix for $n=400$, $\eta=0.01$, Case 1 Activation Function

The results for the configuration with 400 hidden neurons, tanh activation, and a learning rate of 0.01, as detailed in Table 1, demonstrate a highly successful and efficient training process. Figure 1 visually represents this learning dynamic, showing a rapid convergence within the first 10-15 epochs where the average training error decreases and the success rate rises steeply towards its peak. After this initial phase, the learning curve plateaus, indicating the model has effectively converged on a solution, culminating in the near-perfect final training accuracy of 99.47%. Crucially, this robust learning translates exceptionally well to unseen data, as the model achieves an excellent test accuracy of 98.04%. This small gap between training and test performance suggests strong generalization with minimal overfitting. The combination of high accuracy and a very fast average inference time of just 0.1294 ms per pattern confirms that this configuration is a highly effective solution for the digit recognition task.

4.1.2 Second Combination $n=400$, $\eta=0.03$, Case 1 Activation Function

Metric	Value
<i>Configuration Parameters</i>	
Hidden Neurons	400
Activation Case	1 (tanh/tanh)
Learning Rate (η)	0.03
Epochs Run	50
<i>Training Performance</i>	
Total Training Duration	3553.39 s
Average Time per Epoch	71.07 s
Final Training Error	0.3260
Final Training Accuracy	95.87%
<i>Test Performance</i>	
Final Test Error	0.3495
Final Test Accuracy	95.14%
Average Inference Time	0.1343 ms / pattern

Table 2: Performance Metrics for $n=400$, $\eta=0.03$, Case 1 Activation Function

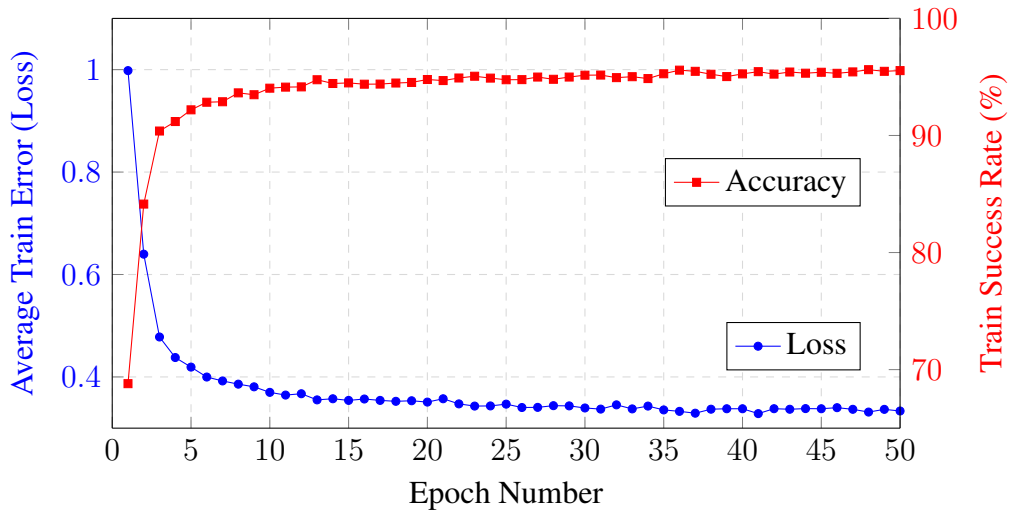


Figure 3: Online Training Performance Loss and Accuracy vs. Epochs for $N=400$, $\eta=0.03$, Case 1.

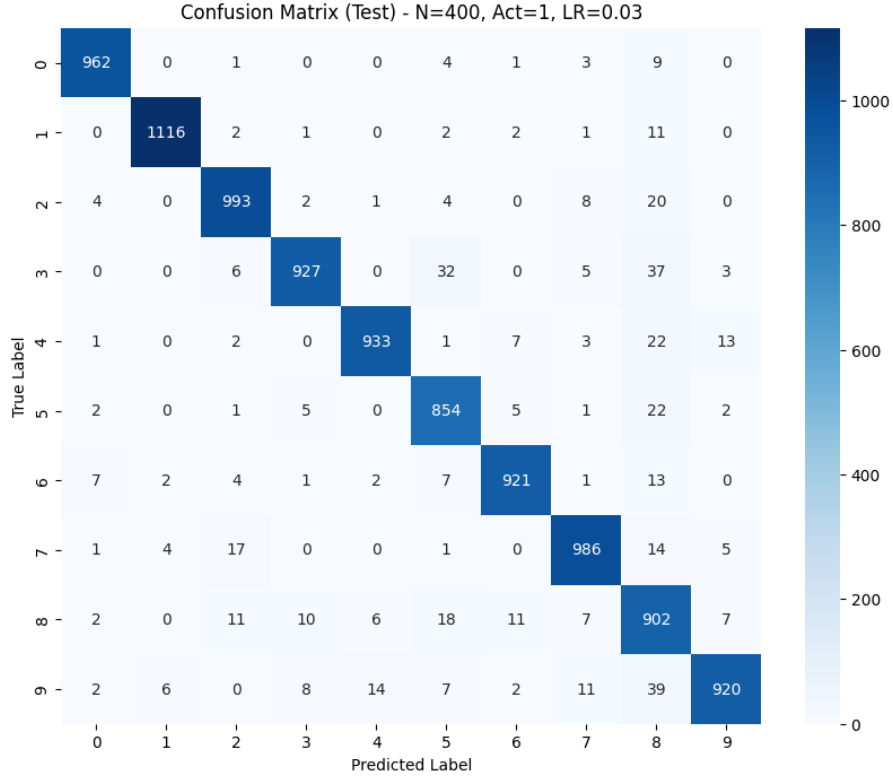


Figure 4: Confusion Matrix for $n=400$, $\eta=0.03$, Case 1 Activation Function

This configuration, utilizing 400 hidden neurons with a higher learning rate of 0.03, results in a noticeably different performance profile compared to the run with a lower learning rate. The model achieves a respectable final test accuracy of 95.14%, as shown in Table 2, but this is a significant drop from the 98% accuracy seen previously. The higher final error values on both the training (0.3260) and test (0.3495) sets are the first indication that this training process was less optimal. Figure 3, the training performance graph, provides a clear visual explanation for this outcome. While the initial learning is extremely rapid in the first 2-3 epochs, the error curve quickly flattens out at a relatively high value (around 0.35) never reaching the low error levels of the previous experiment. This behavior strongly suggests that the learning rate of 0.03 is high rate for this architecture, causing the weight updates to be too large and effectively "oscillating around" the optimal minimum of the error landscape instead of settling into it. The confusion matrix in Figure 4 reveals the practical consequences of this less stable convergence. Although the diagonal is strong, indicating most predictions are correct, the number of misclassifications is significantly higher. Specific patterns of failure are evident: the digit '9' is frequently misclassified as '8' (39 instances), the digit '3' is often mistaken for '8' (37 instances), and the digit '8' is a common incorrect prediction for several true labels (e.g., '2', '3', '9'). This shows that while the model learns the general features of the digits, its ability to distinguish between visually similar characters is compromised by the overly aggressive learning rate. In summary, this configuration demonstrates that a higher learning rate does not always lead to better or faster results; instead, it can hinder the model's ability to fine-tune its parameters, resulting in a lower overall accuracy and more predictable errors.

4.1.3 Third Combination $n=400$, $\eta=0.01$, Case 2 Activation Function

Metric	Value
<i>Configuration Parameters</i>	
Hidden Neurons	400
Activation Case	2 (ReLU/tanh)
Learning Rate (η)	0.01
Epochs Run	50
<i>Training Performance</i>	
Total Training Duration	3502.23 s
Average Time per Epoch	70.04 s
Final Training Error	0.0064
Final Training Accuracy	99.73%
<i>Test Performance</i>	
Final Test Error	0.0597
Final Test Accuracy	98.45%
Average Inference Time	0.1078 ms / pattern

Table 3: Performance Metrics for $n=400$, $\eta=0.01$, Case 2 Activation Function

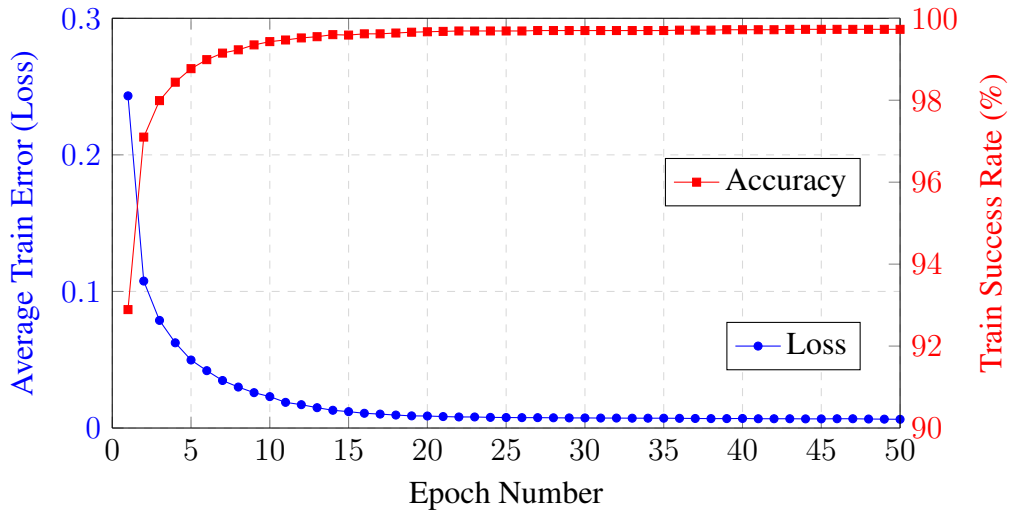


Figure 5: Online Training Performance for $n=400$, $\eta=0.01$, Case 2 Activation Function

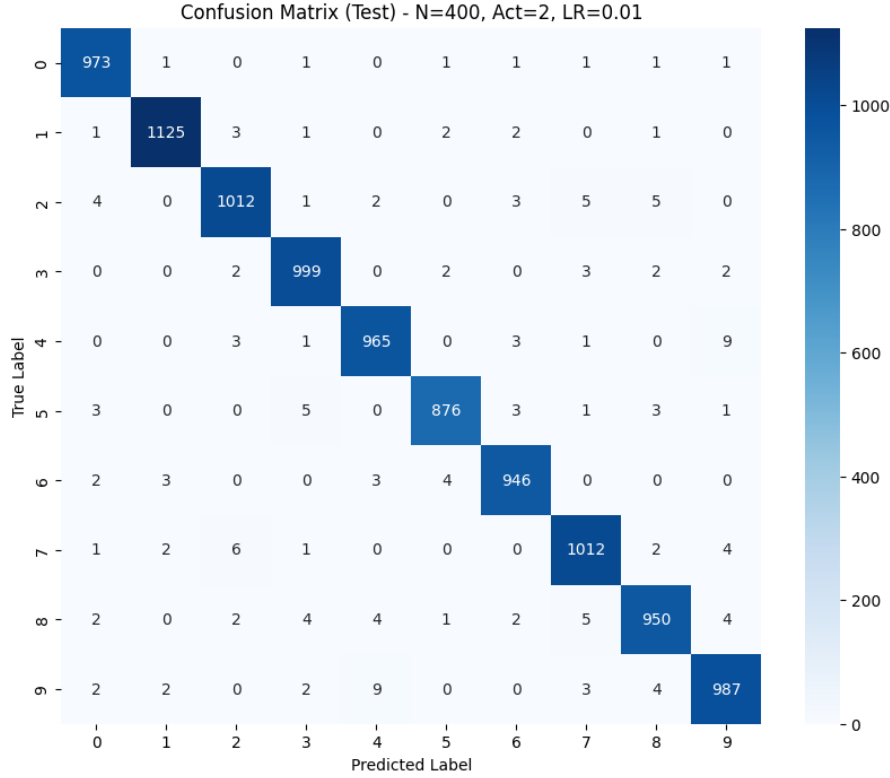


Figure 6: Confusion Matrix for $n=400$, $\eta=0.01$, Case 2 Activation Function

The introduction of the ReLU activation function for the hidden layer marks an important and highly positive shift in performance, establishing this configuration. The key metrics in Table 3 immediately highlight this success: the model achieves an outstanding final test accuracy of 98.45%, coupled with a remarkably low final test error of 0.0597. This represents a significant improvement in both accuracy and error reduction compared to the previous tanh-based models. Furthermore, the slightly lower average time per epoch (70.04 s) suggests that the ReLU function, being computationally simpler than the tanh function, also offers a marginal gain in training efficiency. The training performance graph in Figure 5 provides a compelling visual narrative of this efficiency. The learning process is exceptionally rapid and stable. The average training error decreases from its initial value to below 0.1 within just two epochs, and the accuracy correspondingly surges past 97% in the same time-frame. The curves then converge smoothly and plateau at an extremely low error level, demonstrating that the ReLU activation enables the network to find a much deeper and more accurate minimum in the error landscape, without the instability observed in other configurations. The confusion matrix in Figure 6 confirms the model's exceptional classification power. The diagonal is intensely populated, indicating a very high rate of correct predictions across all digit classes. The off-diagonal cells are sparse and contain very low numbers, showing that misclassifications are rare events. Unlike the previous run with the higher learning rate, there are no single points of catastrophic failure; the most frequent error is a '9' being mistaken for a '4' (9 instances), a visually plausible mistake. In conclusion, switching to the ReLU activation function provided a substantial performance boost, resulting in a model that learns faster, achieves lower error rates, and classifies with significantly higher precision.

4.1.4 Fourth Combination $n=400$, $\eta=0.03$, Case 2 Activation Function

Metric	Value
<i>Configuration Parameters</i>	
Hidden Neurons	400
Activation Case	2 (ReLU/tanh)
Learning Rate (η)	0.03
Epochs Run	50
<i>Training Performance</i>	
Total Training Duration	3624.27 s
Average Time per Epoch	72.49 s
Final Training Error	0.1480
Final Training Accuracy	95.18%
<i>Test Performance</i>	
Final Test Error	0.1675
Final Test Accuracy	94.48%
Average Inference Time	0.1076 ms / pattern

Table 4: Performance Metrics for $n=400$, $\eta=0.03$, Case 2 Activation Function

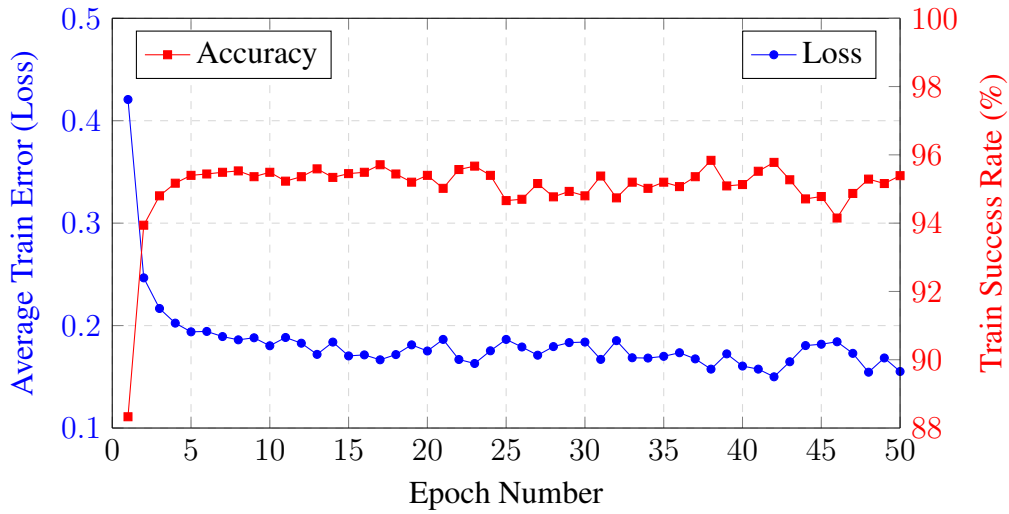


Figure 7: Online Training Performance for $n=400$, $\eta=0.03$, Case 2 Activation Function

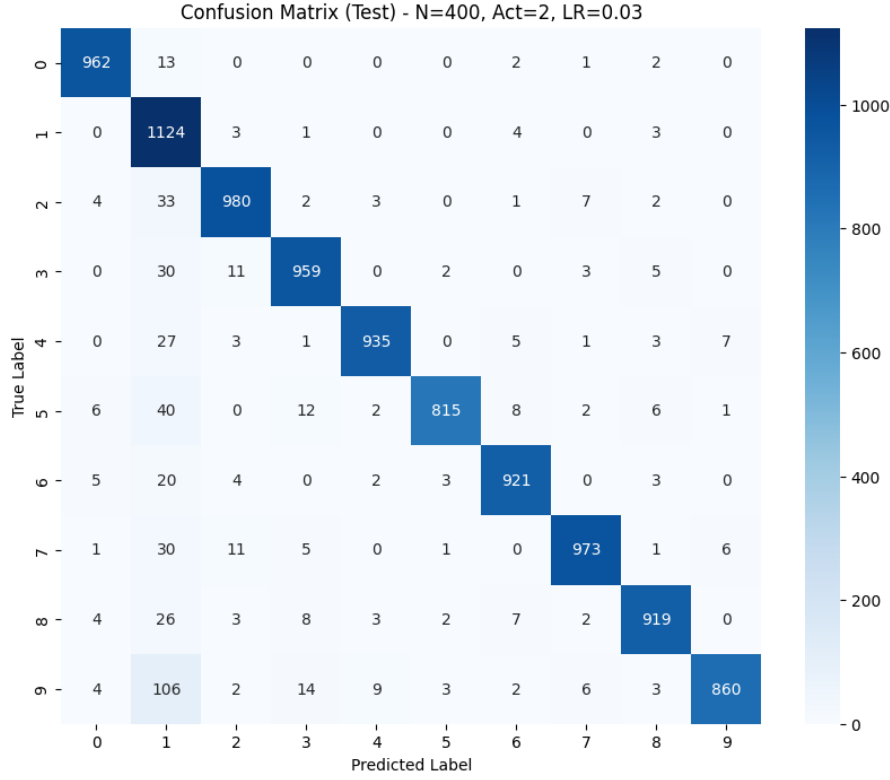


Figure 8: Confusion Matrix for $n=400$, $\eta=0.03$, Case 2 Activation Function

This fourth combination, which pairs the powerful ReLU activation function with a high learning rate of 0.03, serves as a clear example of how an inappropriate hyperparameter can undermine an otherwise strong architecture. As shown in Table 4, the model’s performance drops, achieving a final test accuracy of only 94.48% and a high test error of 0.1675. These results are inferior to those obtained with the same architecture and a lower learning rate, and are comparable to the suboptimal performance observed in the tanh-based model with the same high learning rate. The training graph in Figure 7 visually diagnoses the problem. The plot exhibits classic signs of training instability. Both the error and accuracy curves are oscillating significantly from one epoch to the next without settling into a stable convergence. This unusual behavior suggests that the learning rate is too aggressive, causing the weight updates to consistently overshoot the optimal point in the error landscape, thereby preventing the model from fine-tuning its parameters effectively. The confusion matrix in Figure 8 reveals the severe consequences of this unstable training. While many digits are classified correctly, there is a catastrophic and systematic failure in identifying the digit ‘9’. An alarming 106 instances of ‘9’ are misclassified as ‘1’. This is not a random error, but a fundamental flaw that the model has learned, likely due to overly large weight adjustments. Other digits like ‘5’ and ‘2’ are also frequently mispredicted as ‘1’ (40 and 33 times, respectively), showing a strong bias has developed. In conclusion, this experiment demonstrates that even with a superior activation function like ReLU, an excessively high learning rate can lead to unstable training, poor generalization, and the development of significant, systematic classification errors.

4.1.5 Fifth Combination $n=800$, $\eta=0.01$, Case 1 Activation Function

Metric	Value
<i>Configuration Parameters</i>	
Hidden Neurons	800
Activation Case	1 (tanh/tanh)
Learning Rate (η)	0.01
Epochs Run	50
<i>Training Performance</i>	
Total Training Duration	12017.35 s
Average Time per Epoch	240.35 s
Final Training Error	0.0220
Final Training Accuracy	99.39%
<i>Test Performance</i>	
Final Test Error	0.0855
Final Test Accuracy	98.07%
Average Inference Time	0.2644 ms / pattern

Table 5: Performance Metrics for $n=800$, $\eta=0.01$, Case 1 Activation Function

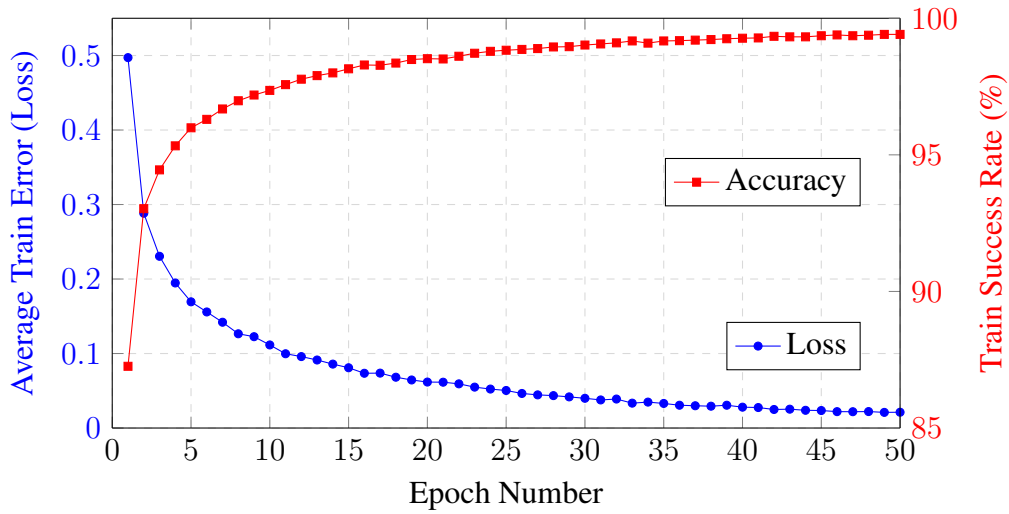


Figure 9: Online Training Performance for $n=800$, $\eta=0.01$, Case 1 Activation Function

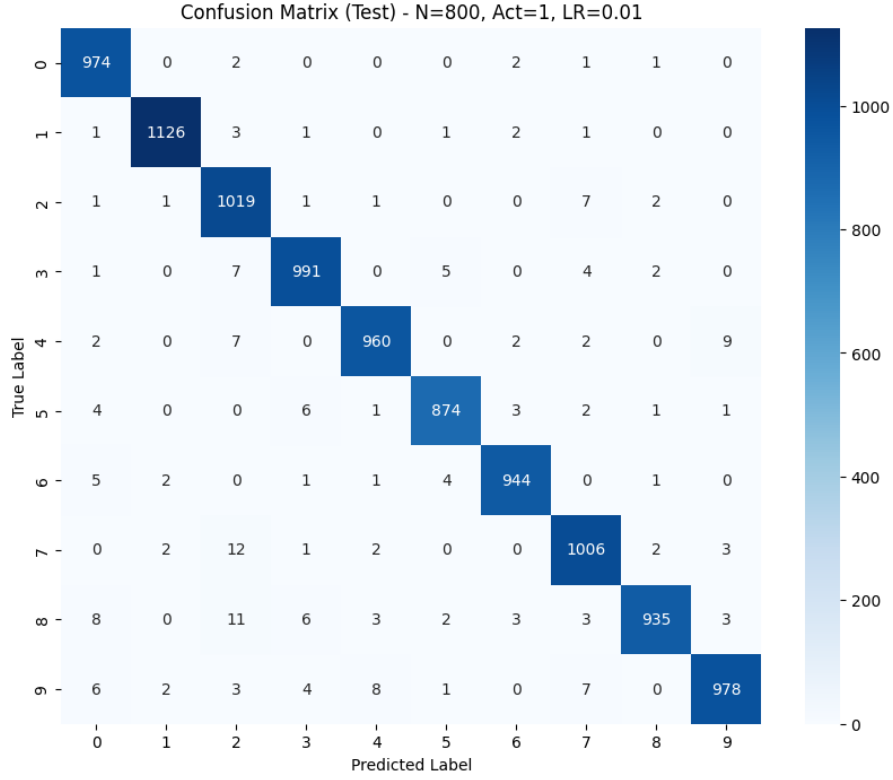


Figure 10: Confusion Matrix for $n=800$, $\eta=0.01$, Case 1 Activation Function

Increasing the network's capacity by expanding the hidden layer to 800 neurons, while retaining the tanh activation function and a stable learning rate of 0.01, yields a highly effective model. The performance metrics in Table 5 show a strong final test accuracy of 98.07%, nearly identical to the best-performing 400-neuron model, indicating that the network has successfully leveraged its increased capacity. As expected, the larger network size results in a significantly longer training time per epoch (240.35 s). The training graph in Figure 9 depicts a smooth and ideal convergence pattern. The error curve shows a steep, consistent decline, while the accuracy curve rises rapidly to a high plateau, indicating a stable learning process free from the volatility associated with a high learning rate. The confusion matrix in Figure 10 complements this by showing a very clean classification performance. The diagonal is heavily populated, and off-diagonal errors are minimal and spread out, with no single, glaring pattern of failure. This demonstrates that the larger model has learned to distinguish between all digit classes effectively, resulting in a robust and high-performing classifier.

4.1.6 Sixth Combination $n=800$, $\eta=0.03$, Case 1 Activation Function

Metric	Value
<i>Configuration Parameters</i>	
Hidden Neurons	800
Activation Case	1 (tanh/tanh)
Learning Rate (η)	0.03
Epochs Run	50
<i>Training Performance</i>	
Total Training Duration	12256.12 s
Average Time per Epoch	245.12 s
Final Training Error	0.6915
Final Training Accuracy	85.33%
<i>Test Performance</i>	
Final Test Error	0.7161
Final Test Accuracy	85.15%
Average Inference Time	0.2613 ms / pattern

Table 6: Performance Metrics for $n=800$, $\eta=0.03$, Case 1 Activation Function

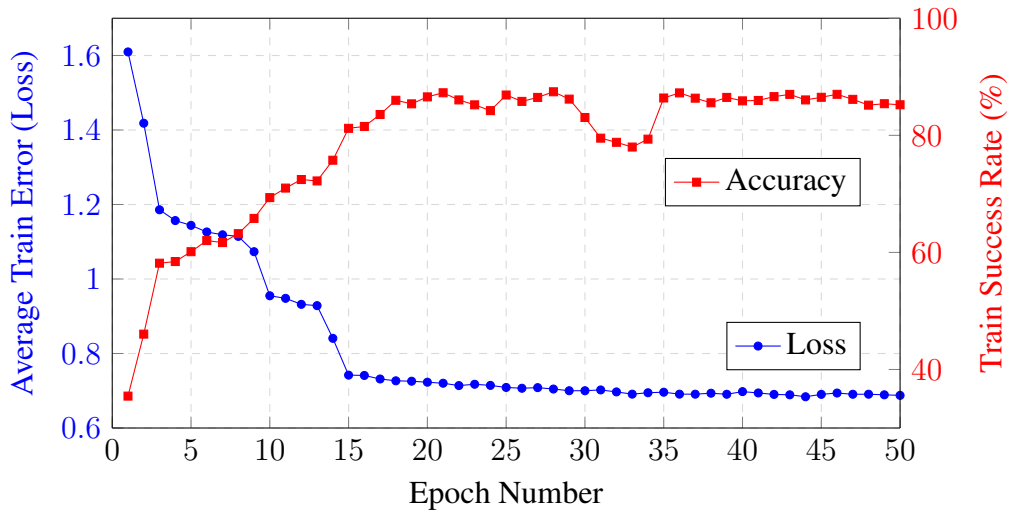


Figure 11: Online Training Performance for $n=800$, $\eta=0.03$, Case 1 Activation Function

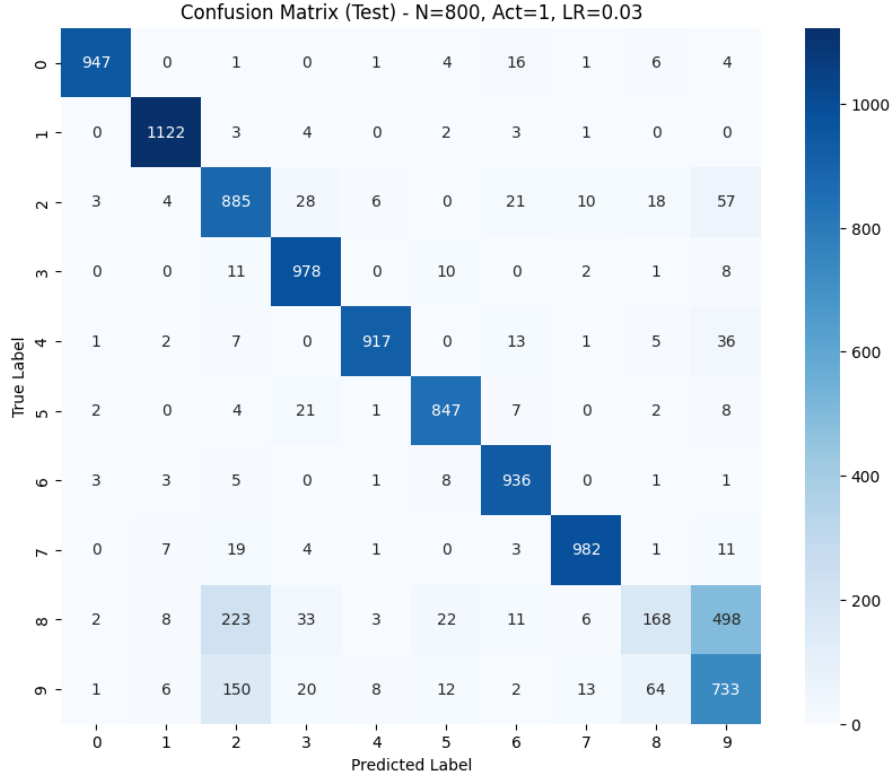


Figure 12: Confusion Matrix for $n=800$, $\eta=0.03$, Case 1 Activation Function

This configuration represents a high failure, demonstrating the severe negative impact of combining a high-capacity model (800 neurons) with an overly aggressive learning rate (0.03). The performance metrics in Table 6 immediately signal a problem, with a poor final test accuracy of only 85.15% and a corresponding test error of 0.7161. Notably, the training accuracy is almost identical, indicating that the model did not overfit; rather, it fundamentally failed to converge to an effective solution for the dataset. Figure 11 provides a clear visual diagnosis of this failure. The training process is extremely unstable. The accuracy curve (in red) struggles to climb and exhibits extreme volatility, with sharp peaks and troughs throughout the 50 epochs, never settling on a high-performance plateau. The error curve (in blue) decreases initially but remains at a very high level, confirming that the learning process is far from an optimal solution. This chaotic behavior is a classic symptom of a learning rate that is too high, causing the weight updates to constantly overshoot the minimum in the error landscape. The devastating consequences of this instability are shown in the confusion matrix in Figure 12. The classification performance is deeply flawed, with systematic errors. The most highlighted failure is the model's inability to correctly identify the digit '8'; an astonishing 223 instances of '8' are misclassified as '2', and another 168 are misclassified as '9'. Similarly, 150 instances of the digit '9' are also incorrectly predicted as '2'. These are not minor mistakes, but fundamental misunderstandings that the model has learned. In conclusion, the combination of a large, complex network and a high learning rate created a volatile training dynamic that prevented convergence, resulting in a poorly performing model with catastrophic, systemic biases in its predictions.

4.1.7 Seventh Combination $n=800$, $\eta=0.03$, Case 1 Activation Function

Metric	Value
<i>Configuration Parameters</i>	
Hidden Neurons	800
Activation Case	2 (ReLU/tanh)
Learning Rate (η)	0.01
Epochs Run	50
<i>Training Performance</i>	
Total Training Duration	9977.28 s
Average Time per Epoch	199.55 s
Final Training Error	0.0047
Final Training Accuracy	99.80%
<i>Test Performance</i>	
Final Test Error	0.0579
Final Test Accuracy	98.50%
Average Inference Time	0.4109 ms / pattern

Table 7: Performance Metrics for $n=800$, $\eta=0.01$, Case 2 Activation Function

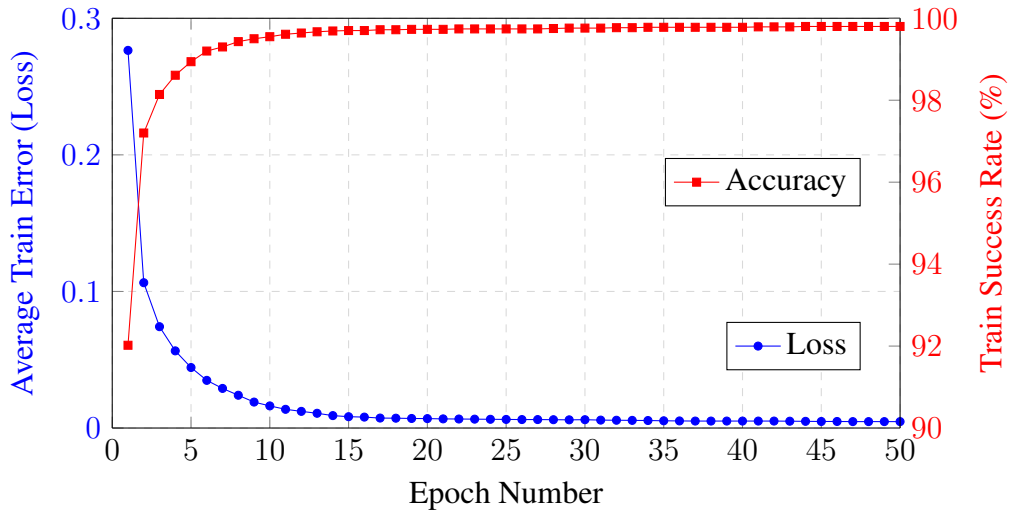


Figure 13: Online Training Performance for $n=800$, $\eta=0.01$, Case 2 Activation Function

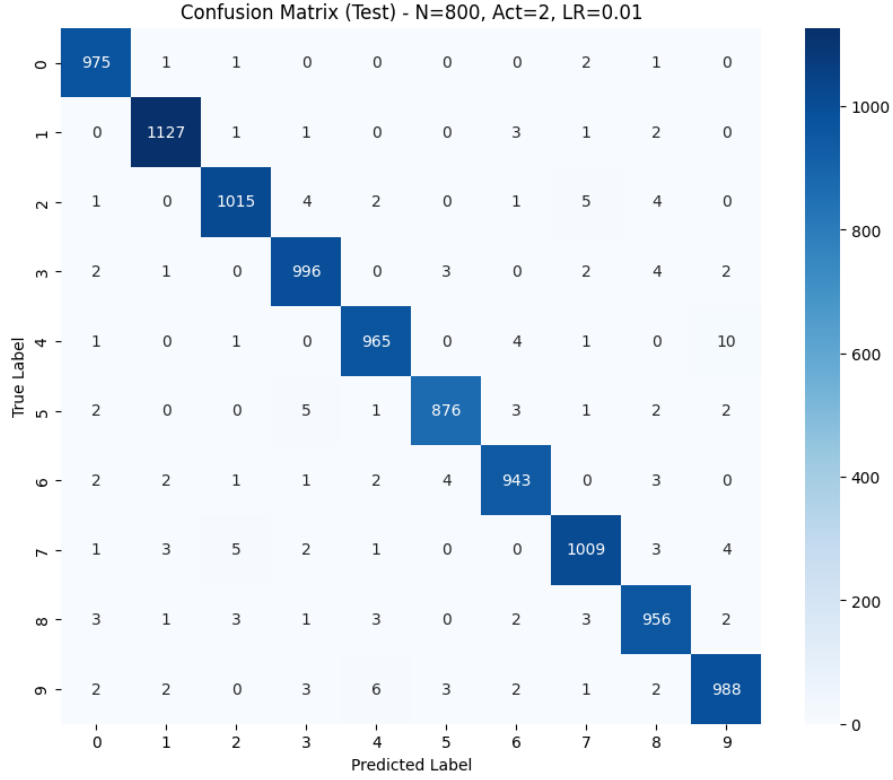


Figure 14: Confusion Matrix for $n=800$, $\eta=0.01$, Case 2 Activation Function

This configuration, which combines the largest network capacity (800 neurons) with the efficient ReLU activation function (Case 2) and a stable learning rate (0.01), represents the peak of performance among the experiments conducted. The performance metrics in Table 7 confirm this, showcasing the highest final test accuracy achieved at 98.50%. This is supported by an extremely low final training error of 0.0047 and a final test error of 0.0579, indicating the model has converged to a highly optimal solution and generalizes exceptionally well to unseen data. Figure 13 provides a visual representation of this ideal training process. The learning curves are smooth and stable, with the average training error reduced rapidly within the first 10-15 epochs while the success rate climbs just as quickly to a high plateau. This demonstrates highly efficient and stable convergence, free from the volatility observed in configurations with higher learning rates. The confusion matrix in Figure 14 offers the final proof of this model's superior classification ability. The matrix is characterized by a dominant, intensely populated diagonal, confirming the high rate of correct predictions across all ten digit classes. The off-diagonal cells are extremely sparse and contain low single-digit numbers, indicating that misclassifications are rare and, importantly, not systematic. In summary, the combination of high network capacity, an effective activation function, and a well-chosen learning rate has produced the most accurate and reliable classifier in this study.

4.1.8 Eighth Combination $n=800$, $\eta=0.03$, Case 2 Activation Function

Metric	Value
<i>Configuration Parameters</i>	
Hidden Neurons	800
Activation Case	2 (ReLU/tanh)
Learning Rate (η)	0.03
Epochs Run	50
<i>Training Performance</i>	
Total Training Duration	10037.42 s
Average Time per Epoch	200.75 s
Final Training Error	0.5233
Final Training Accuracy	76.70%
<i>Test Performance</i>	
Final Test Error	0.5405
Final Test Accuracy	75.94%
Average Inference Time	0.4252 ms / pattern

Table 8: Performance Metrics for $n=800$, $\eta=0.03$, Case 2 Activation Function

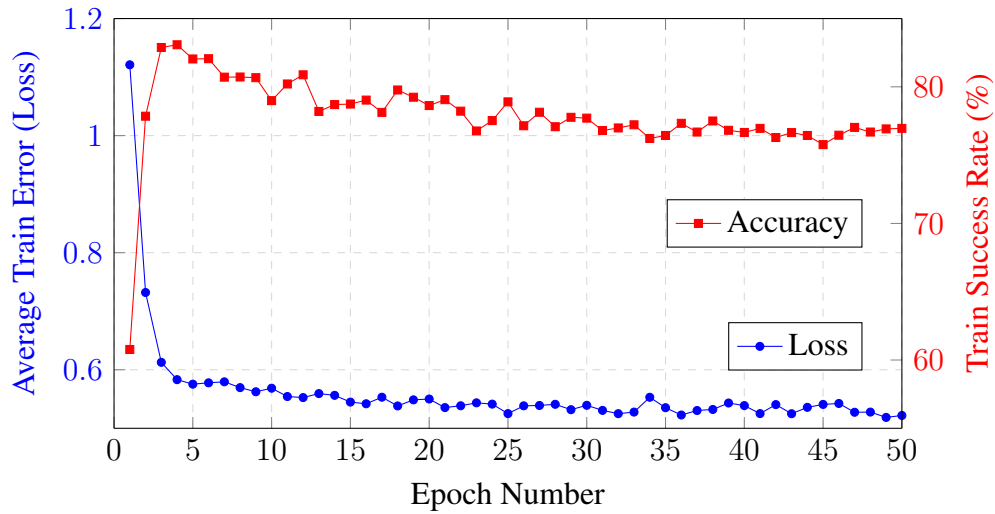


Figure 15: Online Training Performance for $n=800$, $\eta=0.03$, Case 2 Activation Function

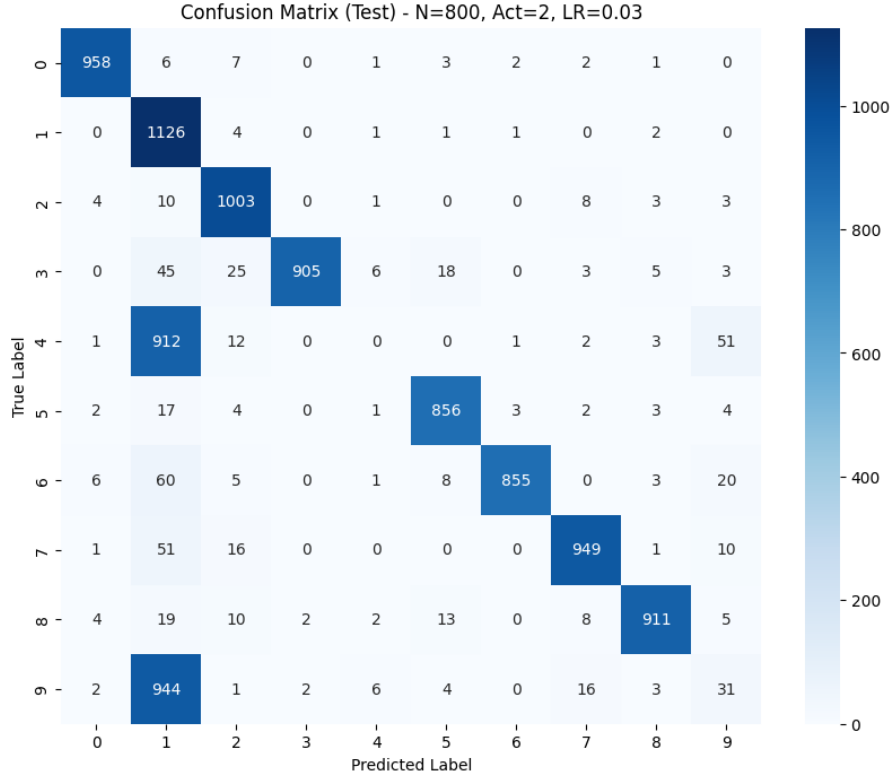


Figure 16: Confusion Matrix for $n=800$, $\eta=0.03$, Case 2 Activation Function

This final configuration, which combines the largest model (800 neurons) and a powerful ReLU activation with the highest learning rate (0.03), represents a training failure and is by far the worst-performing model in the study. The performance metrics in Table 8 are unequivocal, with the final test accuracy decreasing to a very poor 75.94% and the test error soaring to 0.5405. The fact that the training accuracy is similarly low at 76.70% indicates this is not a case of overfitting; rather, the model fundamentally failed to learn the task effectively. Figure 15 illustrates a training instability caused by an excessively high learning rate. The accuracy curve is extremely volatile, jumping and falling erratically without ever achieving a stable, high-performance plateau. Similarly, the loss curve, while decreasing initially, remains high and noisy, "bouncing" around the 0.5 mark. This chaotic behavior shows that the large weight updates are preventing the model from settling into any meaningful minimum in the error landscape. The confusion matrix in Figure 16 reveals the shocking consequences of this unstable training. The model exhibits a catastrophic failure in distinguishing several digits, developing a massive bias towards predicting the digit '1'. An astonishing 912 instances of the digit '4' and an even more staggering 944 instances of the digit '9' are incorrectly classified as '1'. This is not a random pattern of errors, but a systemic flaw that was learned during the chaotic training process. In conclusion, this experiment demonstrates that even with a high-capacity model and a strong activation function, an improperly tuned, aggressive learning rate can completely derail the learning process, resulting in a highly inaccurate and unreliable classifier.

4.2 Overall Analysis of Online Learning Experiments

Configuration	Epochs	Total Time (s)	Avg Epoch (s)	Train Error	Train Acc. (%)	Test Error	Test Acc. (%)
N=400, C=1, L=0.01	50	3707.44	74.15	0.0139	99.47	0.0786	98.04
N=400, C=1, L=0.03	50	3553.39	71.07	0.3260	95.87	0.3495	95.14
N=400, C=2, L=0.01	50	3502.23	70.04	0.0064	99.73	0.0597	98.45
N=400, C=2, L=0.03	50	3624.27	72.49	0.1480	95.18	0.1675	94.48
N=800, C=1, L=0.01	50	12017.35	240.35	0.0220	99.39	0.0855	98.07
N=800, C=1, L=0.03	50	12256.12	245.12	0.6915	85.33	0.7161	85.15
N=800, C=2, L=0.01	50	9977.28	199.55	0.0047	99.80	0.0579	98.50
N=800, C=2, L=0.03	50	10037.42	200.75	0.5233	76.70	0.5405	75.94

Table 9: Summary of Performance Metrics for all 8 Hyperparameter Configurations.

Effect of Hidden Layer Size: Increasing the number of hidden neurons from 400 to 800 provided only a marginal improvement in accuracy when the training was stable. For instance, in the best-performing Case 2 with a learning rate of 0.01, increasing the neuron count from 400 to 800 only lifted the test accuracy from 98.45% to 98.50%. This minor gain came at a significant computational cost, with the average training time per epoch increasing from approximately 70 seconds to over 200 seconds. This suggests that a 400-neuron hidden layer is already sufficiently complex to capture the features of the MNIST dataset, and the larger model offers a poor trade-off between performance and efficiency.

Effect of Activation Function: The choice of activation function proved to be a critical factor. Across all stable configurations (i.e., with $\eta = 0.01$), the models using ReLU for the hidden layer (Case 2) consistently outperformed those using only tanh (Case 1). The ReLU-based models not only achieved slightly higher final test accuracies but also demonstrated faster convergence, reaching lower error levels earlier in the training process. This is evident when comparing the N=800, $\eta = 0.01$ models, where the Case 2 (ReLU) model reached 98.50% accuracy, while the Case 1 (tanh) model reached 98.07%. This performance is likely due to ReLU’s effectiveness at mitigating the vanishing gradient problem, allowing for more efficient learning.

Effect of Learning Rate: The learning rate had the most important impact on performance. A learning rate of $\eta = 0.01$ consistently produced stable training and high-performing models across all architectures, with test accuracies above 98%. In sharp contrast, the higher learning rate of $\eta = 0.03$ was resulted in bad performance. It led to training instability, characterized by volatile error and accuracy curves, and resulted in significantly lower final accuracies (typically below 95% or even 85%). In the most extreme cases (e.g., N=800, Case 2), the high learning rate caused a failure, leading to a massive systematic bias where the model misclassified thousands of digits. This demonstrates that a learning rate of 0.03 was too aggressive, preventing the models from converging to an optimal solution.

Best Configuration Analysis: The best performance was achieved with the configuration: N=800, Case=2 (ReLU/tanh), and $\eta = 0.01$. This model achieved the highest final test accuracy of 98.50% with a final test error of just 0.0579. The training history and confusion matrix for this model are shown in Figure 13 and Figure 14, respectively.

4.3 Best Hyperparameter Combination Used in Batch Learning

4.3.1 Batch size B=20

Metric	Value
<i>Configuration Parameters</i>	
Hidden Neurons	800
Activation Case	2 (ReLU/tanh)
Learning Rate (η)	0.01
Batch Size	20
Epochs Run	50
<i>Training Performance</i>	
Total Training Duration	2158.35 s
Average Time per Epoch	43.17 s
Final Training Error	0.0191
Final Training Accuracy	99.47%
<i>Test Performance</i>	
Final Test Error	0.0816
Final Test Accuracy	98.03%
Average Inference Time	0.2455 ms / pattern

Table 10: Performance Metrics for Batch Learning (B=20) on the Best Configuration.

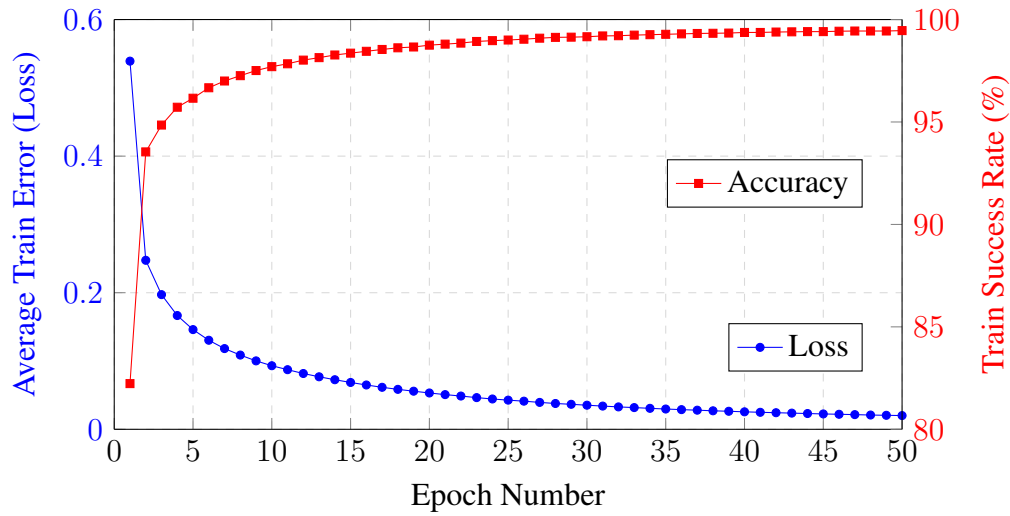


Figure 17: Batch Learning (B=20) Training Performance for N=800, Case=2, LR=0.01.

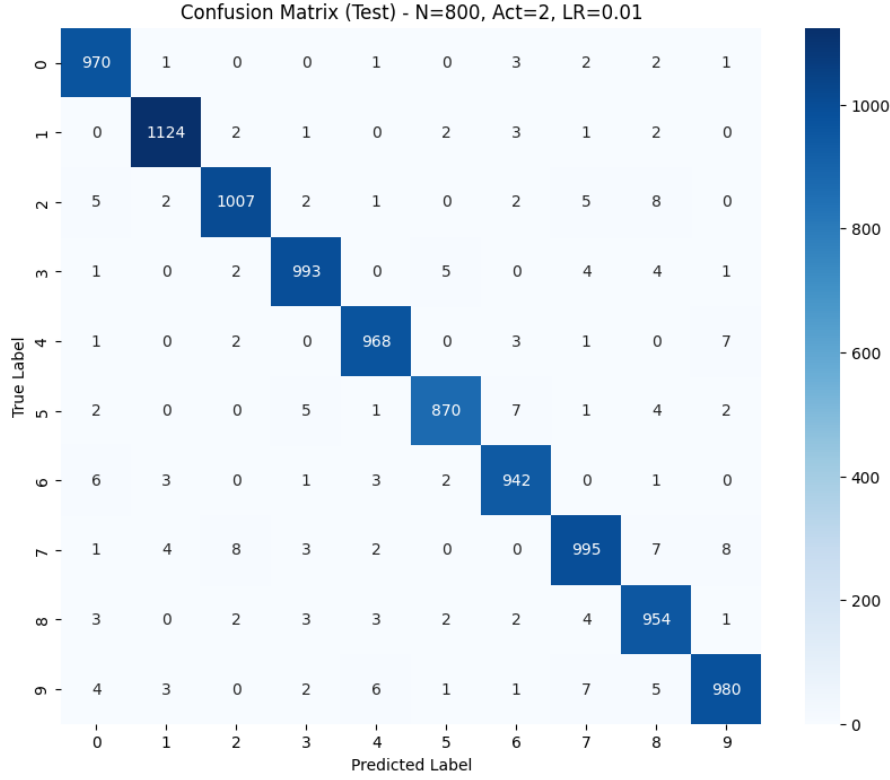


Figure 18: Confusion Matrix for Batch Learning (B=20) on the Best Configuration.

Switching to batch learning with a batch size of 20 on the best-performing architecture presents a clear and important trade-off between computational efficiency and peak performance. As detailed in Table 10, the most dramatic effect is the massive reduction in training time. The total training duration reduced to just 2158.35 seconds which is approximately four times faster than the online equivalent. This speedup is a direct result of performing fewer, more computationally efficient weight updates. This efficiency gain, however, comes at the cost of a slight reduce in performance; the final test accuracy is 98.03%, which is excellent but slightly lower than the 98.50% achieved by the online model. Figure 17, the training performance graph, illustrates that the learning process was very stable and effective, despite being slightly less optimal than the online method. The loss curve shows a smooth, classic exponential decay, and the accuracy curve rises quickly to a high plateau. The smoothness of these curves is characteristic of batch learning, as the averaging of gradients over each batch leads to a less noisy and more direct convergence path compared to the sample by sample updates of online learning. The confusion matrix in Figure 18 confirms that the model is still a very strong and reliable classifier. The diagonal is heavily populated, indicating a high rate of correct predictions. This shows that despite the small drop in overall accuracy, the model did not develop any significant biases and continues to generalize well across all digit classes. In summary, batch learning with B=20 offers a substantial improvement in training speed for a very minor compromise in final accuracy, making it a highly practical and efficient choice for this task.

4.3.2 Batch size B=200

Metric	Value
<i>Configuration Parameters</i>	
Hidden Neurons	800
Activation Case	2 (ReLU/tanh)
Learning Rate (η)	0.01
Batch Size	200
Epochs Run	50
<i>Training Performance</i>	
Total Training Duration	559.49 s
Average Time per Epoch	11.19 s
Final Training Error	0.1306
Final Training Accuracy	96.75%
<i>Test Performance</i>	
Final Test Error	0.1524
Final Test Accuracy	96.25%
Average Inference Time	0.4503 ms / pattern

Table 11: Performance Metrics for Batch Learning (B=200) on the Best Configuration.

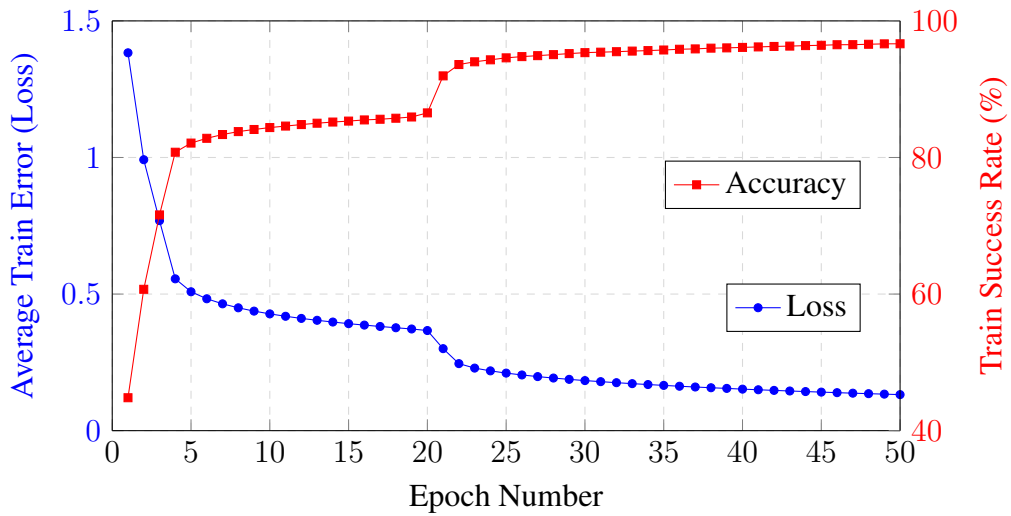


Figure 19: Batch Learning (B=200) Training Performance for N=800, Case=2, LR=0.01.

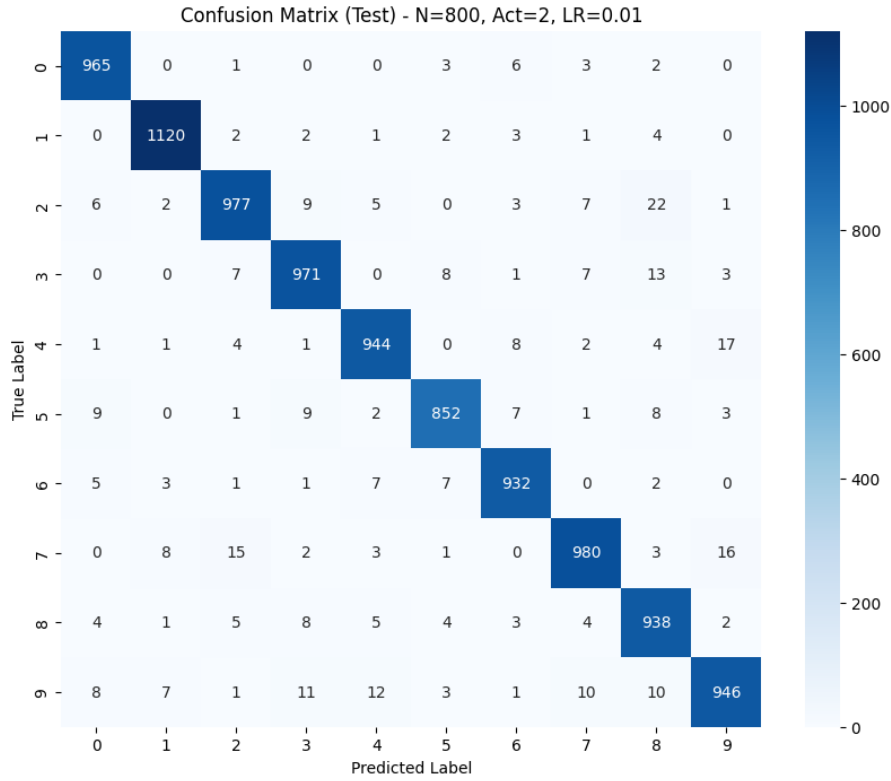


Figure 20: Confusion Matrix for Batch Learning (B=200) on the Best Configuration.

Increasing the batch size to 200 further amplifies the trade-off between training speed and model accuracy observed in the previous batch experiment. The most striking result, as shown in Table 11, is the staggering computational efficiency: the total training duration is just 559.49 seconds, with an average epoch time of only 11.19 seconds. This is nearly four times faster than the B=20 run and almost eighteen times faster than the original online learning model. However, this efficiency comes with a cost, as the final test accuracy drops to 96.25%, the lowest among the three successful learning modes for this architecture. The training performance graph in Figure 19 clearly illustrates why the accuracy is lower. The learning process is visibly slower to converge compared to the smaller batch and online methods. The loss curve decreases much more gradually, and the accuracy curve takes significantly longer to climb, only surpassing the 95% mark after more than 25 epochs. The large batch size means that weight updates are far less frequent and are based on a much broader average of the data. While this creates a very smooth learning curve, it also reduces the stochasticity that helps the model explore the error landscape and settle into a sharp, optimal minimum, causing it to converge to a less ideal solution. A notable jump in performance occurs around epoch 21, where the model appears to overcome a plateau and finds a better gradient, after which learning continues more steadily. The confusion matrix in Figure 20 provides a clear picture of this reduced accuracy. While still performing well overall, misclassifications are more frequent and pronounced than in the B=20 or online models. For example, there are 22 instances of the digit '2' being misclassified as '7', and 15 instances of '7' being mistaken for '2', highlighting a specific confusion pair that the model struggled with. In conclusion, while a large batch size of 200 offers a massive speed advantage, it comes at the cost of slower convergence per epoch and a slight reduction in the final classification accuracy, demonstrating a clear trade-off between computational speed and model precision.

5 Conclusion

This project successfully demonstrated the implementation, and training of a multilayer neural network from scratch for handwritten digit recognition, with a peak test accuracy of 98.50%. The

comprehensive experimental analysis revealed several key insights into the behavior of the network. The learning rate proved to be the most critical hyperparameter, with a value of 0.01 consistently yielding stable convergence and high performance, whereas a rate of 0.03 universally led to training instability and significantly degraded accuracy. The superiority of the ReLU activation function (Case 2) over tanh (Case 1) was clearly established, as it enabled faster convergence to lower error states and consistently produced higher performing models. The analysis also showed that increasing the hidden layer size from 400 to 800 offered diminishing returns, providing only a marginal accuracy boost at a substantial computational cost. Furthermore, the investigation into batch learning highlighted a fundamental trade-off between efficiency and performance; while batch sizes of 20 and 200 dramatically reduced training time, they resulted in slightly lower final accuracies compared to the online learning method. Ultimately, this project serves as a practical confirmation of core neural network principles, illustrating that while a well-chosen architecture is important, careful hyperparameter tuning is leading to unlocking a model's full predictive potential and balancing performance with computational efficiency.